

PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
Graduação em Sistemas de Informação

Igor Palhares
Paulo Henrique

TRABALHO DE ARQUITETURA DE COMPUTADORES - ULA

Belo Horizonte
2019

Igor Palhares
Paulo Henrique

TRABALHO DE ARQUITETURA DE COMPUTADORES - ULA

Trabalho apresentado à disciplina Arquitetura de computadores, do curso de Sistemas de Informação da Pontifícia Universidade Católica de Minas Gerais.

Orientador: Prof. Claudiney Vander Ramos

Belo Horizonte
2019

RESUMO

Este projeto se refere a uma ULA que calcular a soma, subtração, multiplicação, e divisão de números convertidos nas bases: decimal, octal, hexadecimal e binário. A ULA está dividida em quatro tipos: ULAS de 1 bits, 8 bits, 16 bits e 32 bits.

Para a realização deste projeto, utilizamos diversos recursos práticos e teóricos desenvolvidos em salas de aula, além de pesquisas bibliográficas, consultas à fóruns e sites relacionados ao tema.

A proposta é desenvolver uma ULA baseado nas pesquisas realizadas pelo grupo e técnicas de programação orientado a objeto que tem como objetivo a utilização dos conceitos aprendidos em sala de aula e aplica-los para a criação da ULA.

Após a finalização deste projeto, percebemos a importância e a melhoria que tivemos na criação de um sistema da ULA, utilizando a linguagem C#, organizando melhor os códigos produzidos, facilitando manutenções e implementações futuras, além disso, no progresso que tivemos na administração e comunicação entre os membros do grupo.

Palavras-chave: Orientação a Objetos. Desenvolver a ULA. Aplicação prática de arquitetura de Computadores.

Sumário

1. Introdução	5
1.1 Objetivos	5
1.2 Organização do Trabalho	5
1.3 Utilização do Sistema ULA.....	6
2. Desenvolvimento	9
2.1. O que é uma ULA?	9
2.2. Entradas e saídas da ULA	9
2.3. Representação em sinal – magnitude	10
2.4 Representação de inteiros	11
2.5 Portas e álgebra booleana	12
2.6 Explicação da porta lógica And:	12
2.7 Explicação da porta lógica Or:	12
2.8 Explicação da porta lógica Not:.....	12
2.9 Explicação da porta lógica Xor:.....	12
2.10 Explicação da classe ComplementoDe2:	13
2.11 Explicação do somador:	13
2.12 Códigos.....	14
2.12.1 Códigos da classe Decoders.....	14
2.12.1 Explicação dos Códigos da classe Decoders.....	14
2.12.2 Códigos da classe FullAdder.....	15
2.12.2 Explicação dos Códigos da classe FullAdder.....	15
2.12.3 Códigos da classe ConversoesBases.....	16
2.12.3 Explicação dos Códigos da classe ConversoesBases.....	17
2.12.4 Códigos da classe PortasLogicas	18

2.12.4 Explicação dos Códigos da classe PortasLogicas	19
2.12.5 Códigos da classe PortasLogicasDerivadas	20
2.12.5 Explicação dos Códigos da classe PortasLogicasDerivadas	22
2.12.6 Códigos da classe ULAs_Base	22
2.12.6 Explicação dos Códigos da classe ULAs_Base	27
2.12.7 Códigos da classe ULAs_Total.....	28
2.12.7 Explicação dos Códigos da classe ULAs_Total.....	32
3. CONCLUSÃO	33
REFERÊNCIAS	34

1. Introdução

De acordo com o tema, unidade lógica e aritmética (ULA), foram realizadas diversas pesquisas e testes com diferentes ferramentas, com o objetivo de deixar o sistema mais próximo da nossa realidade.

Através das especificações ditas pelo nosso professor de Arquitetura de computadores, criamos um sistema seguindo o que foi visto em sala de aula, relacionado a programação orientada a objeto.

O sistema consiste em calcular a soma, subtração, multiplicação, e divisão de números convertidos nas bases: decimal, octal, hexadecimal e binário. A ULA está dividida em quatro tipos: ULAS de 1 bits, 8 bits, 16 bits e 32 bits. Cada um desses botões será responsável para realizar alguma função e facilitar na utilização do sistema.

1.1 Objetivos

Com o conhecimento obtido através das aulas, pesquisas sobre Arquitetura de computadores, o Trabalho Final (ULA) tem como objetivo desenvolver um sistema que, a partir de diferentes números e ULAS de 1, 8, 16 e 32 bits, realize operações aritméticas que nos mostre o resultado das operações em 4 bases diferentes (decimal, octal, hexadecimal e binário).

1.2 Organização do Trabalho

Começamos o desenvolvimento abordando sobre o funcionamento da ULA, ou seja, a lógica que iríamos utilizar para a criação, logo após mostraremos como se utiliza, a explicação e os codigos do programa, concluimos e apresentamos as referencias.

1.3 Utilização do Sistema ULA

Ao abrir o programa nos deparamos com menu com as seguintes opções:

Selecionar a ULA, colocar dois números na base decimal, selecionar as operações, o botão de conversão, os botões das operações aritméticas e o botão de sair.

The screenshot shows a window titled "Projeto ULA". At the top, it says "Selecione a ULA que deseja usar:" followed by a dropdown menu. Below this, there are two input fields labeled "Num A:" and "Num B:". Between them is a "Converter" button. Underneath these are four more input fields for each number, labeled "Binário:", "Octal:", "Hexadecimal:", and "Decimal:". In the center, there are four buttons for arithmetic operations: "+", "-", "x", and "/". To the right of these are another set of four input fields for the result, also labeled "Binário:", "Octal:", "Hexadecimal:", and "Decimal:". Below the result fields is a section titled "Resposta:" with four more input fields for the result in different bases. At the bottom left is a "Sair" button. At the bottom right, it says "Alunos:" followed by the names "Igor Palhares Reis" and "Paulo Henrique Rocha Ribeiro".

Ao selecionar a ULA, o usuário terá que escolher qual tipo de ULA que gostaria de usar: Ula de 1, 8, 16 ou 32 bits.

Projeto ULA

Selecione a ULA que deseja usar:

Num A: Num B:

Binário: Binário:

Octal: Octal:

Hexadecimal: Hexadecimal:

Decimal: Decimal:

Resposta:

Binário:

Octal:

Hexadecimal:

Decimal:

Sair

Alunos:

Igor Palhares Reis

Paulo Henrique Rocha Ribeiro

Para teste, utilizamos a Ula de 8 bits. Após selecionar a ula, teremos que colocar dois números na base decimal. No nosso teste, utilizamos os números 20 e 10 na base decimal.

O próximo passo é clicar no botão converter, e o sistema irá converter os números inseridos nas bases: decimal, octal, hexadecimal e binário.

Projeto ULA

Selecione a ULA que deseja usar:

ULA de 8 bits

Num A: 20 Num B: 10

Converter

Binário: 00010100 Binário: 00001010

Octal: 24 Octal: 12

Hexadecimal: 14 Hexadecimal: A

Decimal: 20 Decimal: 10

Resposta:

Binário:

Octal:

Hexadecimal:

Decimal:

Sair

Alunos:

Igor Palhares Reis

Paulo Henrique Rocha Ribeiro

Por último, o usuário terá que selecionar o tipo de operação que gostaria de fazer. No nosso teste utilizamos a multiplicação, mas o usuário poderá selecionar outros tipos de operação como: soma, subtração e divisão.

The screenshot shows a window titled "Projeto ULA" with a standard Windows title bar. The main content area has a light gray background and contains the following elements:

- Header:** "Selecione a ULA que deseja usar:" followed by a dropdown menu showing "ULA de 8 bits".
- Input Section:** Two input fields labeled "Num A:" and "Num B:" with values 20 and 10 respectively. A "Converter" button is located between them.
- Operation Section:** Four buttons for arithmetic operations: "+", "-", "x" (highlighted with a blue border), and "/".
- Input Fields for Operation:** Below the operation buttons are input fields for "Binário:", "Octal:", "Hexadecimal:", and "Decimal:" for both numbers. For Num A, the values are 00010100, 24, 14, and 20. For Num B, the values are 00001010, 12, A, and 10.
- Result Section:** A section titled "Resposta:" with output fields for "Binário:", "Octal:", "Hexadecimal:", and "Decimal:". The results shown are 11001000, 310, C8, and 200.
- Footer:** A "Sair" button on the left and the names "Alunos: Igor Palhares Reis" and "Paulo Henrique Rocha Ribeiro" on the right.

Após isto, a ULA irá nos mostrar o resultado da operação e o usuário poderá testar com outros tipos de ula, outros números, outras operações aritméticas ou encerrar o programa.

2. Desenvolvimento

2.1. O que é uma ULA?

A ULA ou Unidade Lógica e Aritmética, é uma das peças fundamentais da CPU/UCP (unidade central de processamento). Ele é um dispositivo que realiza operações lógicas e aritméticas sobre números representados em circuitos lógicos. Ela executa as principais operações lógicas de um computador, tais como a soma, subtração, divisão e multiplicação, além de determinar se um número é positivo ou negativo ou se é zero. Além de executar funções aritméticas, a ULA deve ser capaz de determinar se uma quantidade é menor ou maior que outra e quando quantidades são iguais. Ela pode executar funções lógicas com letras e com números.

As operações simples de uma ULA consistem em:

- Operações aritméticas com inteiros.
- Operações lógicas bit a bit AND, NOT, OR, XOR.
- Operações de deslocamento de bits (deslocamento, rotação por um número específico de bits para esquerda ou direita, com ou sem sinal); deslocamentos podem ser interpretados como multiplicações ou divisões por 2.

2.2. Entradas e saídas da ULA



2.3. Representação em sinal – magnitude

Existem várias convenções alternativas usadas para representar números inteiros negativos e também positivos, todas envolvem o tratamento do bit mais significativo (mais à esquerda) na palavra como um bit de sinal. Se o bit de sinal for 0, o número é positivo, se o bit de sinal for 1, o número é negativo.

$$\begin{array}{ll} + 18 & = 00010010 \\ - 18 & = 10010010 \quad (\text{sinal-magnitude}) \end{array}$$

Existem diversas desvantagens na representação de sinal – magnitude. Uma é que adição e subtração exigem uma consideração dos números e de suas relativas magnitudes para executar a operação exigida. Outra desvantagem é que existem duas representações do 0:

$$\begin{array}{ll} +0_{10} & = 00000000 \\ -0_{10} & = 10000000 \quad (\text{sinal-magnitude}) \end{array}$$

Isso é inconveniente porque é ligeiramente mais difícil de testar se um valor é igual a 0 (uma operação realizada frequentemente nos computadores) do que se houvesse uma única representação. Por causa dessas desvantagens, a representação sinal -magnitude raramente é usada na implementação da parte inteira da ALU. Em vez disso, o esquema mais comum é a representação de complemento a dois.

2.4 Representação de inteiros

No sistema numérico binário, 1 números quaisquer podem ser representados apenas com os dígitos zero e um, o sinal de menos e a vírgula, ou vírgula fracionada.

$$-1101,0101_2 = -13,3125_{10}$$

Para as finalidades de armazenamento e processamento no computador, porém, não temos o benefício dos sinais de menos e vírgulas. Somente dígitos binários (0 e 1) podem ser usados para representar os números. Se estivermos limitados a inteiros não negativos, a representação é direta.

Uma palavra de 8 bits pode representar os números de 0 a 255, incluindo

$$00000000 = 0$$

$$00000001 = 1$$

$$00101001 = 41$$

$$10000000 = 128$$

$$11111111 = 255$$

Em geral, se uma sequência de n bits de dígitos binários a_{n-1} a a_{n-2} ... a a_1 a a_0 for interpretada como um inteiro sem sinal A , seu valor é:

$$A = \sum_{i=0}^{n-1} 2^i a_i$$

2.5 Portas e álgebra booleana

Circuitos digitais podem ser construídos com um pequeno número de elementos primitivos combinando-os de inúmeras maneiras.

2.6 Explicação da porta lógica And:

Esta porta, recebe dois ou quatro valores, sendo eles falsos ou verdadeiros, e realiza uma operação lógica (and) sobre eles e retorna um valor lógico verdadeiro quando os dois (ou quatro) valores são verdadeiros.

2.7 Explicação da porta lógica Or:

Esta porta, recebe dois, quatro ou oito valores, e realizam uma operação lógica (or) sobre eles, retornando verdadeiro quando pelo menos um dos valores é verdadeiro.

2.8 Explicação da porta lógica Not:

Esta porta, possui um valor que retorna um valor oposto ao número.

2.9 Explicação da porta lógica Xor:

Esta porta, realiza uma operação lógica (and) sobre o valor da operação lógica (not) do primeiro número e o segundo número original, então, realiza-se uma operação (and) sobre o valor da operação lógica (not) do segundo número e o primeiro número original, finalmente, retorna-se à operação lógica (or) das duas operações feitas acima.

2.10 Explicação da classe ComplementoDe2:

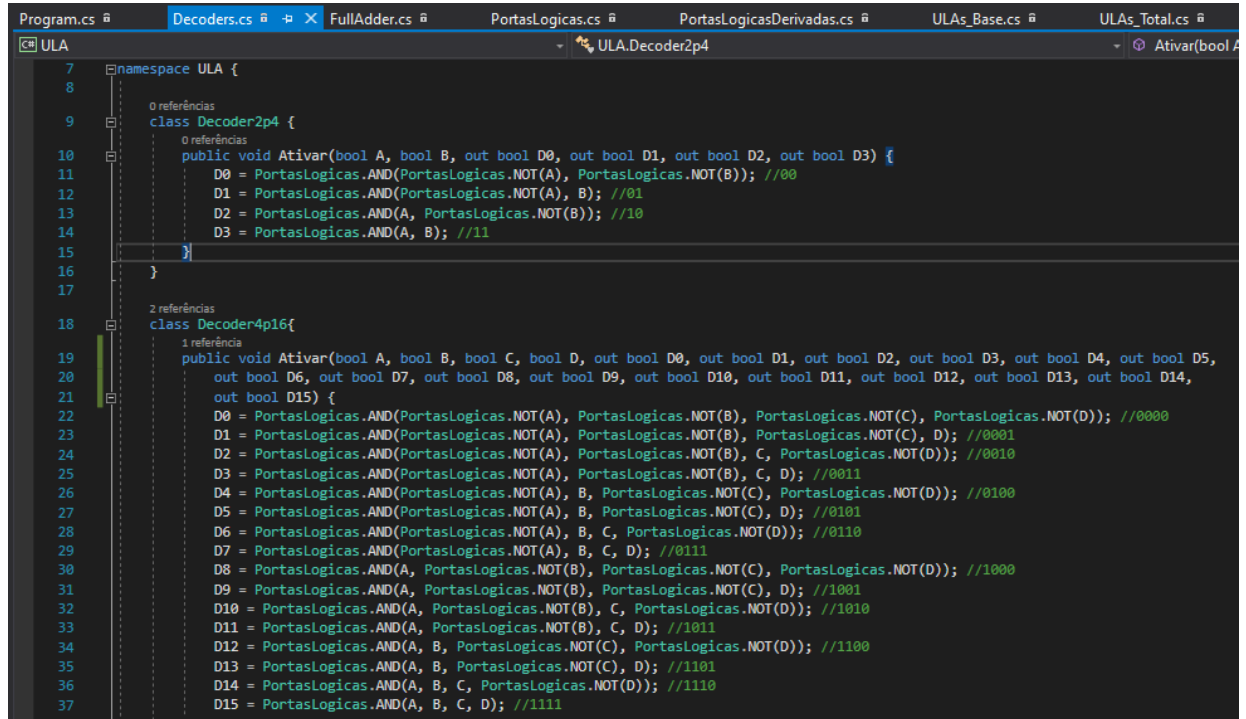
É semelhante ao somador só que específica para fazer o complemento de dois na subtração.

2.11 Explicação do somador:

O somador, tem como objetivo, auxiliar na implementação da unidade lógica e aritmética de 1 bit, ela é a responsável por somar os valores de dois operandos recebidos por parâmetro.

2.12 Códigos

2.12.1 Códigos da classe Decoders



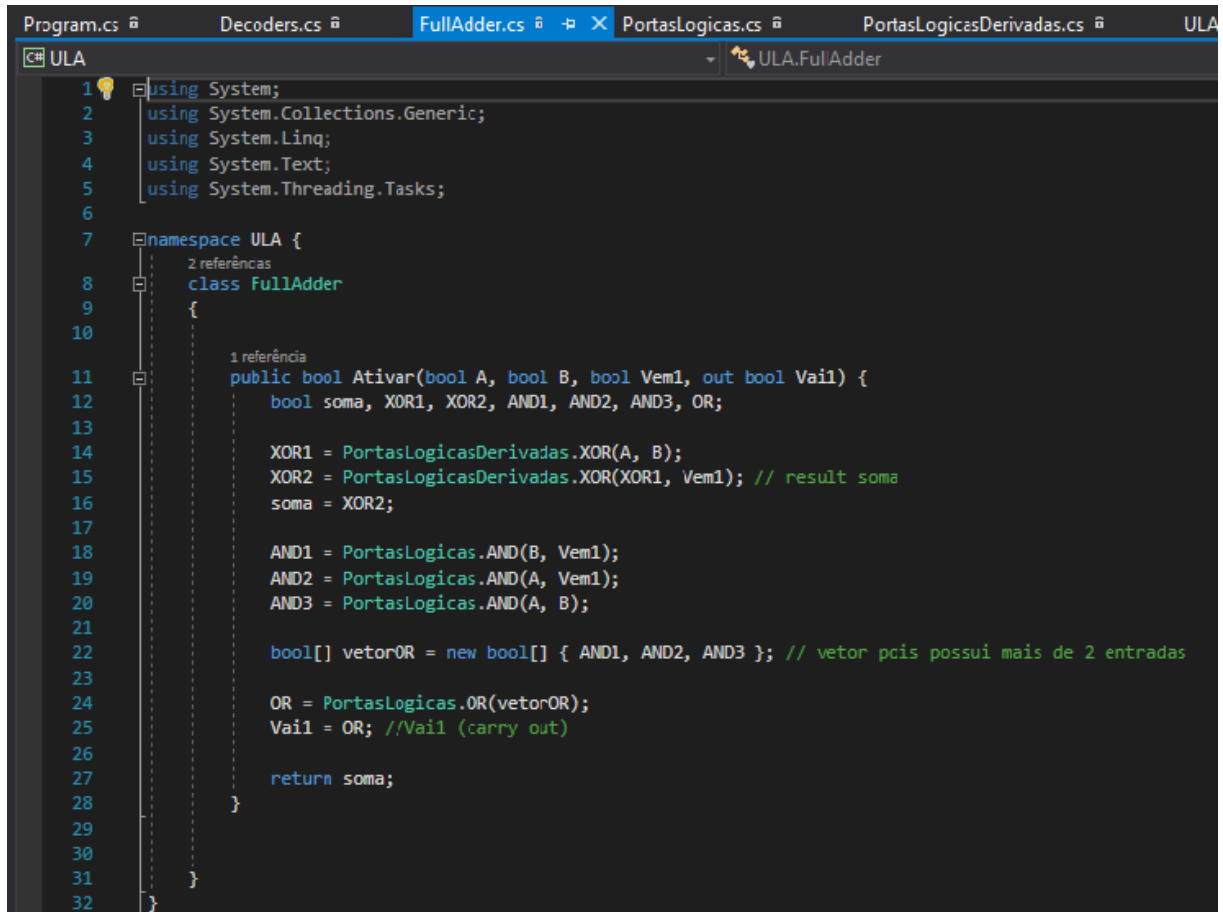
```
7 namespace ULA {
8
9     0 referências
10    class Decoder2p4 {
11        0 referências
12        public void Ativar(bool A, bool B, out bool D0, out bool D1, out bool D2, out bool D3) {
13            D0 = PortasLogicas.AND(PortasLogicas.NOT(A), PortasLogicas.NOT(B)); //00
14            D1 = PortasLogicas.AND(PortasLogicas.NOT(A), B); //01
15            D2 = PortasLogicas.AND(A, PortasLogicas.NOT(B)); //10
16            D3 = PortasLogicas.AND(A, B); //11
17        }
18    }
19
20    2 referências
21    class Decoder4p16{
22        1 referência
23        public void Ativar(bool A, bool B, bool C, bool D, out bool D0, out bool D1, out bool D2, out bool D3, out bool D4, out bool D5,
24            out bool D6, out bool D7, out bool D8, out bool D9, out bool D10, out bool D11, out bool D12, out bool D13, out bool D14,
25            out bool D15) {
26            D0 = PortasLogicas.AND(PortasLogicas.NOT(A), PortasLogicas.NOT(B), PortasLogicas.NOT(C), PortasLogicas.NOT(D)); //0000
27            D1 = PortasLogicas.AND(PortasLogicas.NOT(A), PortasLogicas.NOT(B), PortasLogicas.NOT(C), D); //0001
28            D2 = PortasLogicas.AND(PortasLogicas.NOT(A), PortasLogicas.NOT(B), C, PortasLogicas.NOT(D)); //0010
29            D3 = PortasLogicas.AND(PortasLogicas.NOT(A), PortasLogicas.NOT(B), C, D); //0011
30            D4 = PortasLogicas.AND(PortasLogicas.NOT(A), B, PortasLogicas.NOT(C), PortasLogicas.NOT(D)); //0100
31            D5 = PortasLogicas.AND(PortasLogicas.NOT(A), B, PortasLogicas.NOT(C), D); //0101
32            D6 = PortasLogicas.AND(PortasLogicas.NOT(A), B, C, PortasLogicas.NOT(D)); //0110
33            D7 = PortasLogicas.AND(PortasLogicas.NOT(A), B, C, D); //0111
34            D8 = PortasLogicas.AND(A, PortasLogicas.NOT(B), PortasLogicas.NOT(C), PortasLogicas.NOT(D)); //1000
35            D9 = PortasLogicas.AND(A, PortasLogicas.NOT(B), PortasLogicas.NOT(C), D); //1001
36            D10 = PortasLogicas.AND(A, PortasLogicas.NOT(B), C, PortasLogicas.NOT(D)); //1010
37            D11 = PortasLogicas.AND(A, PortasLogicas.NOT(B), C, D); //1011
38            D12 = PortasLogicas.AND(A, B, PortasLogicas.NOT(C), PortasLogicas.NOT(D)); //1100
39            D13 = PortasLogicas.AND(A, B, PortasLogicas.NOT(C), D); //1101
40            D14 = PortasLogicas.AND(A, B, C, PortasLogicas.NOT(D)); //1110
41            D15 = PortasLogicas.AND(A, B, C, D); //1111
42        }
43    }
44 }
```

2.12.1 Explicação dos Códigos da classe Decoders

O arquivo Decoders.cs possui 2 classes, a classe "Decoder2p4" e a classe "Decoder4p16" que fazem a função de um decoder com 2 entradas e 4 saídas, e 4 entradas e 16 saídas com o método "Ativar", respectivamente (utilizando a classe de PortasLogicas que foi feita).

Resumindo, o decoder é um circuito combinatório, que tem o papel do inverso do codificador, isto é, converte um código binário de entrada (natural, BCD,...) de N bits de entrada em M linha de saída (em que N pode ser qualquer inteiro e M é um inteiro menor ou igual a 2^N), de modo que cada linha de saída será ativada por uma única combinação das possíveis de entrada.

2.12.2 Códigos da classe FullAdder



```
1 using System;
2 using System.Collections.Generic;
3 using System.Linq;
4 using System.Text;
5 using System.Threading.Tasks;
6
7 namespace ULA {
8     class FullAdder
9     {
10
11         public bool Ativar(bool A, bool B, bool Vem1, out bool Vai1) {
12             bool soma, XOR1, XOR2, AND1, AND2, AND3, OR;
13
14             XOR1 = PortasLogicasDerivadas.XOR(A, B);
15             XOR2 = PortasLogicasDerivadas.XOR(XOR1, Vem1); // result soma
16             soma = XOR2;
17
18             AND1 = PortasLogicas.AND(B, Vem1);
19             AND2 = PortasLogicas.AND(A, Vem1);
20             AND3 = PortasLogicas.AND(A, B);
21
22             bool[] vetorOR = new bool[] { AND1, AND2, AND3 }; // vetor pois possui mais de 2 entradas
23
24             OR = PortasLogicas.OR(vetorOR);
25             Vai1 = OR; //Vai1 (carry out)
26
27             return soma;
28         }
29     }
30 }
31
32 }
```

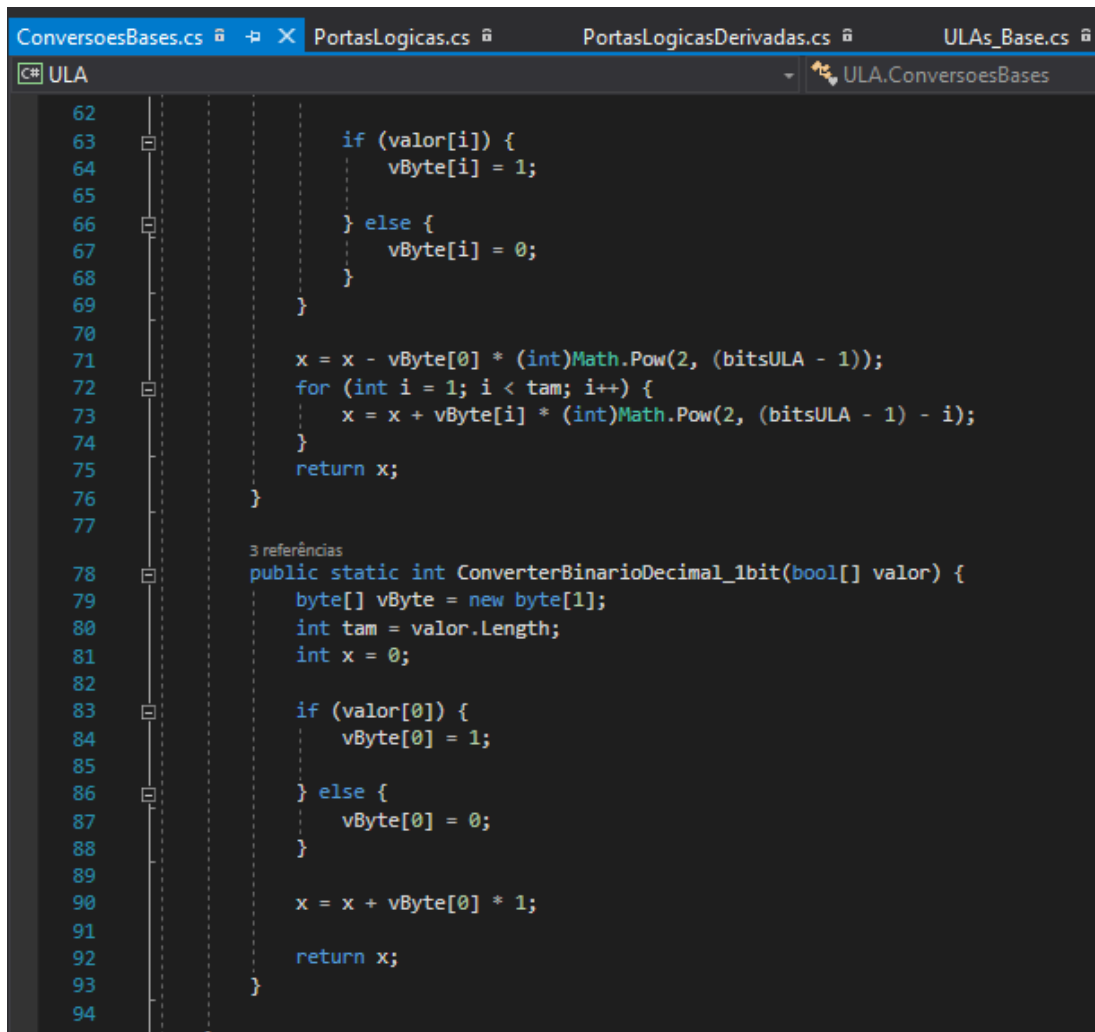
2.12.2 Explicação dos Códigos da classe FullAdder

A classe FullAdder possui nela o método "Ativar" que basicamente contem o circuito de Somador, de modo que quando chamado o método "Ativar" ele faz uma soma levando em consideração 2 valores (a serem somados), o Vem1 e o Vai1.

2.12.3 Códigos da classe ConversoesBases

```
ConversoesBases.cs  PortasLogicas.cs  PortasLogicasDerivadas.cs  ULAs_Base.cs  ULAs_Total.cs
ULA
7 namespace ULA {
8     class ConversoesBases {
9
10         40 referências
11         public static void ConverterDecimalBinario(bool[] vResposta, long num) {
12             long x = num;
13
14             for (int i = 0; i < vResposta.Length; i++) {
15                 if ((x % 2) == 1) {
16                     vResposta[(vResposta.Length - 1) - i] = true;
17                 } else {
18                     vResposta[(vResposta.Length - 1) - i] = false;
19                 }
20                 x = x / 2;
21             }
22         }
23
24         16 referências
25         public static void Converter_Bool_Binario_byte(byte[] vetor, bool[] vetorB, out long y, int tam) {
26             byte i;
27             long x;
28             for (i = 0; i < tam; i++) {
29                 if (vetorB[i]) {
30                     vetor[i] = 1;
31                 } else {
32                     vetor[i] = 0;
33                 }
34             }
35             x = 0;
36             //if(Vai1)x = x - 1 * (int)Math.Pow(2, (tam));
37             for (i = 0; i < tam; i++) {
38                 x = x + vetor[i] * (long)Math.Pow(2, (tam-1) - (i));
```

```
ConversoesBases.cs  PortasLogicas.cs  PortasLogicasDerivadas.cs  ULAs_Base.cs  ULAs_Total.cs
ULA
35         x = 0;
36         //if(Vai1)x = x - 1 * (int)Math.Pow(2, (tam));
37         for (i = 0; i < tam; i++) {
38             x = x + vetor[i] * (long)Math.Pow(2, (tam-1) - (i));
39         }
40         y = x;
41     }
42
43     4 referências
44     public static void Converter_Bool_Binario_byte_1bit(byte[] vetor, bool[] vetorB, out long y) {
45         long x;
46         if (vetorB[0]) {
47             vetor[0] = 1;
48         } else {
49             vetor[0] = 0;
50         }
51         x = 0;
52         x = x + vetor[0] * 1;
53         y = x;
54     }
55
56     9 referências
57     public static int ConverterBinarioDecimal(bool[] valor, int bitsULA) {
58         byte[] vByte = new byte[bitsULA];
59         int tam = valor.Length;
60         int x = 0;
61
62         for (int i = 0; i < tam; i++) {
63             if (valor[i]) {
64                 vByte[i] = 1;
65             } else {
66                 vByte[i] = 0;
67             }
```



```
62
63     if (valor[i]) {
64         vByte[i] = 1;
65     }
66     } else {
67         vByte[i] = 0;
68     }
69 }
70
71 x = x - vByte[0] * (int)Math.Pow(2, (bitsULA - 1));
72 for (int i = 1; i < tam; i++) {
73     x = x + vByte[i] * (int)Math.Pow(2, (bitsULA - 1) - i);
74 }
75 return x;
76 }
77
78 3 referências
79 public static int ConverterBinarioDecimal_1bit(bool[] valor) {
80     byte[] vByte = new byte[1];
81     int tam = valor.Length;
82     int x = 0;
83
84     if (valor[0]) {
85         vByte[0] = 1;
86     } else {
87         vByte[0] = 0;
88     }
89
90     x = x + vByte[0] * 1;
91
92     return x;
93 }
94
95 }
```

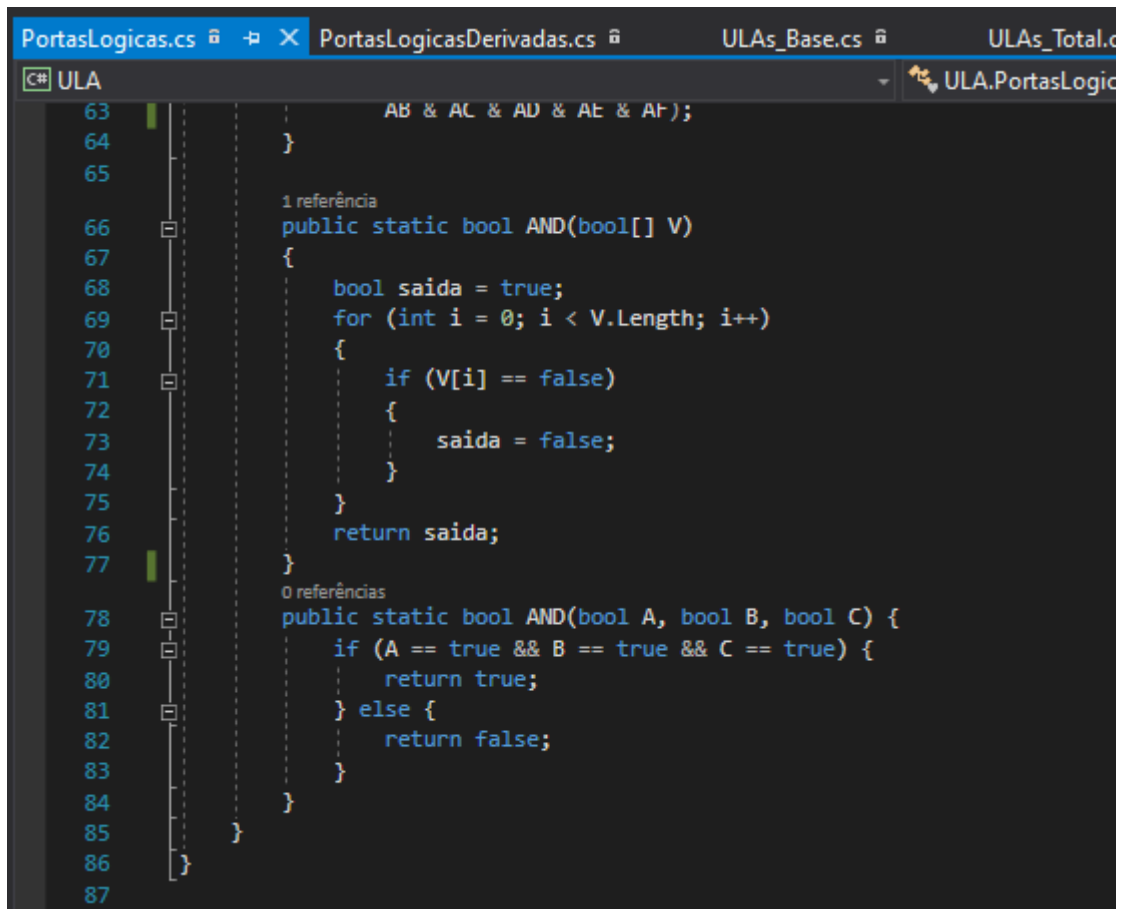
2.12.3 Explicação dos Códigos da classe ConversoesBases

A classe "ConversoesBases" possui diversos métodos de conversão de base que são úteis ao longo do projeto, método para converter de um vetor de booleano, para um vetor de byte, método para converter um vetor de byte, para um número em formato decimal e vice-versa e assim sucessivamente.

2.12.4 Códigos da classe PortasLogicas

```
PortasLogicas.cs  PortasLogicasDerivadas.cs  ULAs_Base.cs  ULAs_Total.cs
C# ULA
6
7 namespace ULA {
8     85 referências
9     class PortasLogicas
10    {
11        //Portas Lógicas com 2 entradas apenas (mais comum)
12        3 referências
13        public static bool OR(bool A, bool B) {
14            return (A || B);
15        }
16
17        18 referências
18        public static bool AND(bool A, bool B)
19        {
20            return (A & B);
21        }
22
23        41 referências
24        public static bool NOT(bool A)
25        {
26            return (!A);
27        }
28
29        //Portas Lógicas com vetor para receber varias entradas (polimorfismo).
30        2 referências
31        public static bool OR(bool[] V)
32        {
33            bool saida = false;
34            for (int i = 0; i < V.Length; i++) {
35                if (V[i] == true) {
36                    saida = true;
37                }
38            }
39            return saida;
40        }
41    }
42 }
```

```
PortasLogicas.cs  PortasLogicasDerivadas.cs  ULAs_Base.cs  ULAs_Total.cs
C# ULA
37
38 1 referência
39 public static bool OR(bool A, bool B, bool C, bool D, bool E, bool F, bool G, bool H, bool I, bool J) {
40     if (A == true || B == true || C == true || D == true || E == true || F == true || G == true || H == true || I == true || J == true) {
41         return true;
42     } else {
43         return false;
44     }
45 }
46
47 16 referências
48 public static bool AND(bool A, bool B, bool C, bool D) {
49     return (A & B & C & D);
50 }
51
52 1 referência
53 public static bool AND(bool A, bool B, bool C, bool D, bool E, bool F, bool G, bool H) {
54     return (A & B & C & D & E & F & G & H);
55 }
56
57 1 referência
58 public static bool AND(bool A, bool B, bool C, bool D, bool E, bool F, bool G, bool H, bool I, bool J, bool K, bool L, bool M,
59     bool N, bool O, bool P) {
60     return (A & B & C & D & E & F & G & H & I & J & K & L & M & N & O & P);
61 }
62
63 1 referência
64 public static bool AND(bool A, bool B, bool C, bool D, bool E, bool F, bool G, bool H, bool I, bool J, bool K, bool L,
65     bool M, bool N, bool O, bool P, bool Q, bool R, bool S, bool T, bool U, bool V, bool W, bool X, bool Y, bool Z,
66     bool AA, bool AB, bool AC, bool AD, bool AE, bool AF) {
67     return (A & B & C & D & E & F & G & H & I & J & K & L & M & N & O & P & Q & R & S & T & U & V & W & X & Y & Z & AA &
68         AB & AC & AD & AE & AF);
69 }
70 }
```

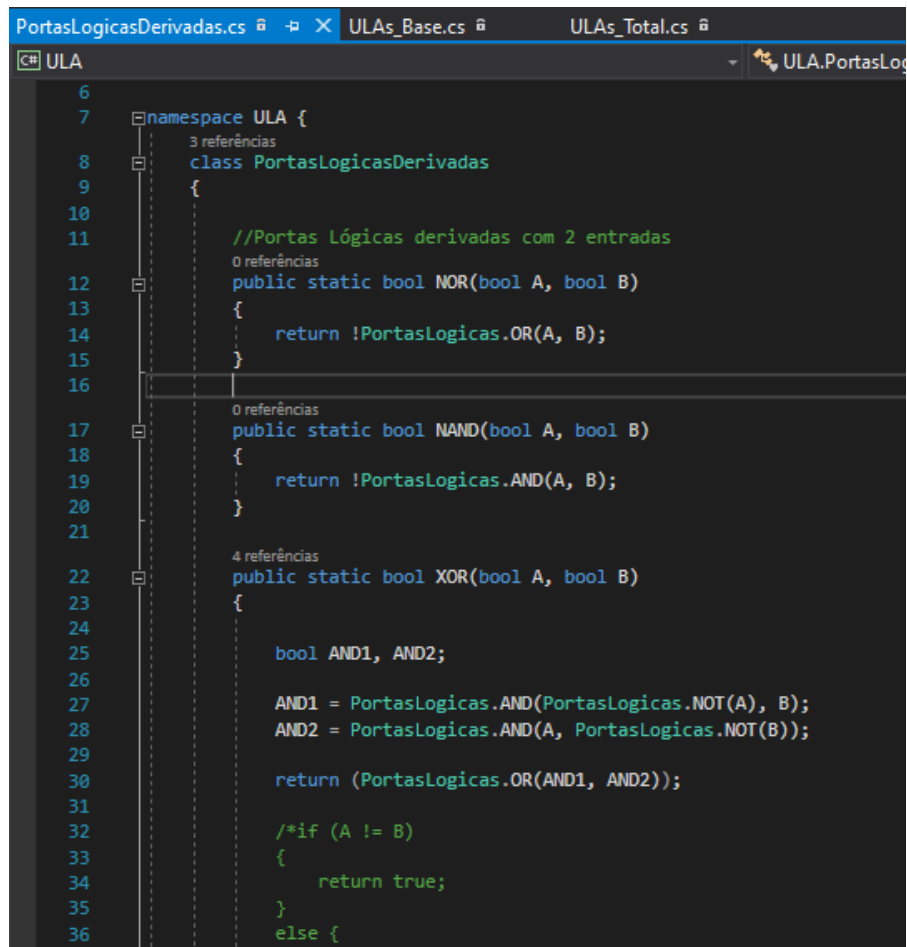


```
PortasLogicas.cs  PortasLogicasDerivadas.cs  ULAs_Base.cs  ULAs_Total.c
C# ULA
63         AB & AC & AD & AE & AF);
64     }
65
66     1 referência
67     public static bool AND(bool[] V)
68     {
69         bool saida = true;
70         for (int i = 0; i < V.Length; i++)
71         {
72             if (V[i] == false)
73             {
74                 saida = false;
75             }
76         }
77         return saida;
78     }
79     0 referências
80     public static bool AND(bool A, bool B, bool C) {
81         if (A == true && B == true && C == true) {
82             return true;
83         } else {
84             return false;
85         }
86     }
87 }
```

2.12.4 Explicação dos Códigos da classe PortasLogicas

A classe "PortasLogicas" como o próprio nome sugere, possui todas as portas lógicas "originais" (AND, OR, NOT) com diversas entradas, pois diversas partes do projeto, foram utilizadas portas lógicas com quantidade de entradas diferentes, além de possuir as portas lógicas que recebem também vetores de boolean.

2.12.5 Códigos da classe PortasLogicasDerivadas



```
6
7 namespace ULA {
8     3 referências
9     class PortasLogicasDerivadas
10    {
11        //Portas Lógicas derivadas com 2 entradas
12        0 referências
13        public static bool NOR(bool A, bool B)
14        {
15            return !PortasLogicas.OR(A, B);
16        }
17
18        0 referências
19        public static bool NAND(bool A, bool B)
20        {
21            return !PortasLogicas.AND(A, B);
22        }
23
24        4 referências
25        public static bool XOR(bool A, bool B)
26        {
27            bool AND1, AND2;
28
29            AND1 = PortasLogicas.AND(PortasLogicas.NOT(A), B);
30            AND2 = PortasLogicas.AND(A, PortasLogicas.NOT(B));
31
32            return (PortasLogicas.OR(AND1, AND2));
33
34            /*if (A != B)
35            {
36                return true;
37            }
38            else {
```

```
PortasLogicasDerivadas.cs  X ULA_Base.cs  ULA_Total.cs
C# ULA
36         else {
37             return false;
38         }*/
39     }
40
41     0 referências
42     public static bool XNOR(bool A, bool B)
43     {
44         return !XOR(A, B);
45     }
46
47     //Portas Lógicas derivadas com vetor para receber varias entradas (polimorfismo).
48
49     0 referências
50     public static bool NOR(bool[] V)
51     {
52         return !PortasLogicas.OR(V);
53     }
54
55     0 referências
56     public static bool NAND(bool[] V)
57     {
58         return !PortasLogicas.AND(V);
59     }
60
61     1 referência
62     public static bool XOR(bool[] V)
63     {
64         bool aux, saida = false;
65         aux = V[0];
66
67         for (int i = 0; i < V.Length; i++) {
68             if (V[i] != aux) {
69                 saida = true;
70             }
71         }
72     }
73 }
```

```
PortasLogicasDerivadas.cs  X ULA_Base.cs  ULA_Total.cs
C# ULA
55         return !PortasLogicas.AND(V);
56     }
57
58     1 referência
59     public static bool XOR(bool[] V)
60     {
61         bool aux, saida = false;
62         aux = V[0];
63
64         for (int i = 0; i < V.Length; i++) {
65             if (V[i] != aux) {
66                 saida = true;
67             }
68         }
69         return saida;
70     }
71
72     0 referências
73     public static bool XNOR(bool[] V)
74     {
75         return !XOR(V);
76     }
77 }
78 }
```

2.12.5 Explicação dos Códigos da classe

PortasLogicasDerivadas

A classe "PortasLogicasDerivadas" como o próprio nome sugere, possui todas as portas lógicas "derivadas" (NAND, NOR, XNOR) com diversas entradas, pois diversas partes do projeto, foram utilizadas portas lógicas com quantidade de entradas diferentes, além de possuir as portas lógicas que recebem também vetores de boolean.

2.12.6 Códigos da classe ULAs_Base

```
ULAs_Base.cs  x ULAs_Totals.cs  x
ULA
7 namespace ULA {
8     99+ referências
9     class ULA_Base {
10         FullAdder Adder = new FullAdder();
11         Decoder4p16 decoder = new Decoder4p16();
12
13
14     57 referências
15     public bool Ativar(bool A, bool B, bool[] F, bool Vem1, out bool Vail) {
16
17         bool AND1, OR1, XOR1, NOT1, NOT2, soma, AND2, AND3, AND4, AND5, AND6, AND7, AND8, S;
18         bool D0, D1, D2, D3, D4, D5, D6, D7, D8, D9, D10, D11, D12, D13, D14, D15;
19
20         AND1 = PortasLogicas.AND(A, B); //P0
21         OR1 = PortasLogicas.OR(A, B); //P1
22         XOR1 = PortasLogicasDerivadas.XOR(A, B); //P1a
23         NOT1 = PortasLogicas.NOT(A); //P2a
24         NOT2 = PortasLogicas.NOT(B); //P2
25
26         soma = Adder.Ativar(A, B, Vem1, out Vail);
27         decoder.Ativar(F[3], F[2], F[1], F[0], out D0, out D1, out D2, out D3, out D4, out D5, out D6, out D7, out D8, out D9, out D10, out D11, out D12, out D13, out D14, out D15);
28
29         AND2 = PortasLogicas.AND(AND1, D0);
30         AND3 = PortasLogicas.AND(OR1, D1);
31         AND4 = PortasLogicas.AND(XOR1, D2);
32         AND5 = PortasLogicas.AND(NOT1, D3);
33         AND6 = PortasLogicas.AND(NOT2, D4);
34         AND7 = PortasLogicas.AND(soma, D5);
35         AND8 = PortasLogicas.AND(PortasLogicas.NOT(XOR1), D7);
36
37         S = PortasLogicas.OR(AND2, AND3, AND4, AND5, AND6, AND7, false, AND8, false, false); // Porta D6 "0110" usada na subtração
38                                         // Porta D8 "1000" usada na multiplicação
39                                         // Porta D9 "1001" usada na divisão
39         return S;
40     }
```

```

ULAs_Base.cs  X  ULAs_Total.cs  ULA.ULa_1Bit_Base
ULA
66  ULA_1Bit_Base ULA5 = new ULA_1Bit_Base();
67  ULA_1Bit_Base ULA6 = new ULA_1Bit_Base();
68  ULA_1Bit_Base ULA7 = new ULA_1Bit_Base();
69
70  6 referências
71  public bool Ativar(bool[] A, bool[] B, bool[] F, bool[] vResp) {
72
73      bool Vail, Vem1, V1 = false;
74
75      vResp[7] = ULA7.Ativar(A[7], B[7], F, V1, out Vail); //0 primeiro CarryIn (Vem1), é sempre false
76      Vem1 = Vail;
77      vResp[6] = ULA6.Ativar(A[6], B[6], F, Vem1, out Vail);
78      Vem1 = Vail;
79      vResp[5] = ULA5.Ativar(A[5], B[5], F, Vem1, out Vail);
80      Vem1 = Vail;
81      vResp[4] = ULA4.Ativar(A[4], B[4], F, Vem1, out Vail);
82      Vem1 = Vail;
83      vResp[3] = ULA3.Ativar(A[3], B[3], F, Vem1, out Vail);
84      Vem1 = Vail;
85      vResp[2] = ULA2.Ativar(A[2], B[2], F, Vem1, out Vail);
86      Vem1 = Vail;
87      vResp[1] = ULA1.Ativar(A[1], B[1], F, Vem1, out Vail);
88      Vem1 = Vail;
89      vResp[0] = ULA0.Ativar(A[0], B[0], F, Vem1, out Vail);
90      Vem1 = Vail;
91
92      return Vail;
93  }
94
95  2 referências
96  class ULA_16Bit {
97
98      ULA_1Bit_Base ULA0 = new ULA_1Bit_Base();

```

```

ULAs_Base.cs  X  ULAs_Total.cs  ULA.ULa_1Bit_Base
ULA
96  2 referências
97  class ULA_16Bit {
98
99      ULA_1Bit_Base ULA0 = new ULA_1Bit_Base();
100     ULA_1Bit_Base ULA1 = new ULA_1Bit_Base();
101     ULA_1Bit_Base ULA2 = new ULA_1Bit_Base();
102     ULA_1Bit_Base ULA3 = new ULA_1Bit_Base();
103     ULA_1Bit_Base ULA4 = new ULA_1Bit_Base();
104     ULA_1Bit_Base ULA5 = new ULA_1Bit_Base();
105     ULA_1Bit_Base ULA6 = new ULA_1Bit_Base();
106     ULA_1Bit_Base ULA7 = new ULA_1Bit_Base();
107     ULA_1Bit_Base ULA8 = new ULA_1Bit_Base();
108     ULA_1Bit_Base ULA9 = new ULA_1Bit_Base();
109     ULA_1Bit_Base ULA10 = new ULA_1Bit_Base();
110     ULA_1Bit_Base ULA11 = new ULA_1Bit_Base();
111     ULA_1Bit_Base ULA12 = new ULA_1Bit_Base();
112     ULA_1Bit_Base ULA13 = new ULA_1Bit_Base();
113     ULA_1Bit_Base ULA14 = new ULA_1Bit_Base();
114     ULA_1Bit_Base ULA15 = new ULA_1Bit_Base();
115
116  6 referências
117  public bool Ativar(bool[] A, bool[] B, bool[] F, bool[] vResp) {
118
119      bool Vail, Vem1, V1 = false;
120
121      vResp[15] = ULA15.Ativar(A[15], B[15], F, V1, out Vail); //0 primeiro CarryIn (Vem1), é sempre false
122      Vem1 = Vail;
123      vResp[14] = ULA14.Ativar(A[14], B[14], F, Vem1, out Vail);
124      Vem1 = Vail;
125      vResp[13] = ULA13.Ativar(A[13], B[13], F, Vem1, out Vail);
126      Vem1 = Vail;
127      vResp[12] = ULA12.Ativar(A[12], B[12], F, Vem1, out Vail);
128      Vem1 = Vail;
129      vResp[11] = ULA11.Ativar(A[11], B[11], F, Vem1, out Vail);

```



```
ULAs_Base.cs  X  ULAs_Total.cs  ULA.ULA_1Bit_Base
[CS] ULA
126     Vem1 = Vail;
127     vResp[11] = ULA11.Ativar(A[11], B[11], F, Vem1, out Vail);
128     Vem1 = Vail;
129     vResp[10] = ULA10.Ativar(A[10], B[10], F, Vem1, out Vail);
130     Vem1 = Vail;
131     vResp[9] = ULA9.Ativar(A[9], B[9], F, Vem1, out Vail);
132     Vem1 = Vail;
133     vResp[8] = ULA8.Ativar(A[8], B[8], F, Vem1, out Vail);
134     Vem1 = Vail;
135     vResp[7] = ULA7.Ativar(A[7], B[7], F, Vem1, out Vail);
136     Vem1 = Vail;
137     vResp[6] = ULA6.Ativar(A[6], B[6], F, Vem1, out Vail);
138     Vem1 = Vail;
139     vResp[5] = ULA5.Ativar(A[5], B[5], F, Vem1, out Vail);
140     Vem1 = Vail;
141     vResp[4] = ULA4.Ativar(A[4], B[4], F, Vem1, out Vail);
142     Vem1 = Vail;
143     vResp[3] = ULA3.Ativar(A[3], B[3], F, Vem1, out Vail);
144     Vem1 = Vail;
145     vResp[2] = ULA2.Ativar(A[2], B[2], F, Vem1, out Vail);
146     Vem1 = Vail;
147     vResp[1] = ULA1.Ativar(A[1], B[1], F, Vem1, out Vail);
148     Vem1 = Vail;
149     vResp[0] = ULA0.Ativar(A[0], B[0], F, Vem1, out Vail);
150     Vem1 = Vail;
151
152     return Vail;
153
154 }
155
156
157 2 referências
158 class ULA_32Bit {
```

```
ULAs_Base.cs  X  ULAs_Total.cs  ULA.ULA_1Bit_Base
[CS] ULA
157 class ULA_32Bit {
158
159     ULA_1Bit_Base ULA0 = new ULA_1Bit_Base();
160     ULA_1Bit_Base ULA1 = new ULA_1Bit_Base();
161     ULA_1Bit_Base ULA2 = new ULA_1Bit_Base();
162     ULA_1Bit_Base ULA3 = new ULA_1Bit_Base();
163     ULA_1Bit_Base ULA4 = new ULA_1Bit_Base();
164     ULA_1Bit_Base ULA5 = new ULA_1Bit_Base();
165     ULA_1Bit_Base ULA6 = new ULA_1Bit_Base();
166     ULA_1Bit_Base ULA7 = new ULA_1Bit_Base();
167     ULA_1Bit_Base ULA8 = new ULA_1Bit_Base();
168     ULA_1Bit_Base ULA9 = new ULA_1Bit_Base();
169     ULA_1Bit_Base ULA10 = new ULA_1Bit_Base();
170     ULA_1Bit_Base ULA11 = new ULA_1Bit_Base();
171     ULA_1Bit_Base ULA12 = new ULA_1Bit_Base();
172     ULA_1Bit_Base ULA13 = new ULA_1Bit_Base();
173     ULA_1Bit_Base ULA14 = new ULA_1Bit_Base();
174     ULA_1Bit_Base ULA15 = new ULA_1Bit_Base();
175     ULA_1Bit_Base ULA16 = new ULA_1Bit_Base();
176     ULA_1Bit_Base ULA17 = new ULA_1Bit_Base();
177     ULA_1Bit_Base ULA18 = new ULA_1Bit_Base();
178     ULA_1Bit_Base ULA19 = new ULA_1Bit_Base();
179     ULA_1Bit_Base ULA20 = new ULA_1Bit_Base();
180     ULA_1Bit_Base ULA21 = new ULA_1Bit_Base();
181     ULA_1Bit_Base ULA22 = new ULA_1Bit_Base();
182     ULA_1Bit_Base ULA23 = new ULA_1Bit_Base();
183     ULA_1Bit_Base ULA24 = new ULA_1Bit_Base();
184     ULA_1Bit_Base ULA25 = new ULA_1Bit_Base();
185     ULA_1Bit_Base ULA26 = new ULA_1Bit_Base();
186     ULA_1Bit_Base ULA27 = new ULA_1Bit_Base();
187     ULA_1Bit_Base ULA28 = new ULA_1Bit_Base();
188     ULA_1Bit_Base ULA29 = new ULA_1Bit_Base();
189     ULA_1Bit_Base ULA30 = new ULA_1Bit_Base();
190     ULA_1Bit_Base ULA31 = new ULA_1Bit_Base();
```

```
ULAs_Base.cs X ULAs_Total.cs
C# ULA ULA.ULA_1Bit_Base

190 ULA_1Bit_Base ULA31 = new ULA_1Bit_Base();
191
192 6 referências
193 public bool Ativar(bool[] A, bool[] B, bool[] F, bool[] vResp) {
194     bool Vail, Vem1, V1 = false;
195
196     vResp[31] = ULA31.Ativar(A[31], B[31], F, V1, out Vail); //O primeiro CarryIn (Vem1), é sempre false
197     Vem1 = Vail;
198     vResp[30] = ULA30.Ativar(A[30], B[30], F, Vem1, out Vail);
199     Vem1 = Vail;
200     vResp[29] = ULA29.Ativar(A[29], B[29], F, Vem1, out Vail);
201     Vem1 = Vail;
202     vResp[28] = ULA28.Ativar(A[28], B[28], F, Vem1, out Vail);
203     Vem1 = Vail;
204     vResp[27] = ULA27.Ativar(A[27], B[27], F, Vem1, out Vail);
205     Vem1 = Vail;
206     vResp[26] = ULA26.Ativar(A[26], B[26], F, Vem1, out Vail);
207     Vem1 = Vail;
208     vResp[25] = ULA25.Ativar(A[25], B[25], F, Vem1, out Vail);
209     Vem1 = Vail;
210     vResp[24] = ULA24.Ativar(A[24], B[24], F, Vem1, out Vail);
211     Vem1 = Vail;
212     vResp[23] = ULA23.Ativar(A[23], B[23], F, Vem1, out Vail);
213     Vem1 = Vail;
214     vResp[22] = ULA22.Ativar(A[22], B[22], F, Vem1, out Vail);
215     Vem1 = Vail;
216     vResp[21] = ULA21.Ativar(A[21], B[21], F, Vem1, out Vail);
217     Vem1 = Vail;
218     vResp[20] = ULA20.Ativar(A[20], B[20], F, Vem1, out Vail);
219     Vem1 = Vail;
220     vResp[19] = ULA19.Ativar(A[19], B[19], F, Vem1, out Vail);
221     Vem1 = Vail;
222     vResp[18] = ULA18.Ativar(A[18], B[18], F, Vem1, out Vail);
223     Vem1 = Vail;
```

```
ULAs_Base.cs  X  ULAs_Total.cs  ULA.ULA_1Bit_Base
C# ULA
222 vResp[16] = ULA16.Ativar(A[16], B[16], F, Vem1, out Vai1);
223 Vem1 = Vai1;
224 vResp[17] = ULA17.Ativar(A[17], B[17], F, Vem1, out Vai1);
225 Vem1 = Vai1;
226 vResp[16] = ULA16.Ativar(A[16], B[16], F, Vem1, out Vai1);
227 Vem1 = Vai1;
228 vResp[15] = ULA15.Ativar(A[15], B[15], F, Vem1, out Vai1);
229 Vem1 = Vai1;
230 vResp[14] = ULA14.Ativar(A[14], B[14], F, Vem1, out Vai1);
231 Vem1 = Vai1;
232 vResp[13] = ULA13.Ativar(A[13], B[13], F, Vem1, out Vai1);
233 Vem1 = Vai1;
234 vResp[12] = ULA12.Ativar(A[12], B[12], F, Vem1, out Vai1);
235 Vem1 = Vai1;
236 vResp[11] = ULA11.Ativar(A[11], B[11], F, Vem1, out Vai1);
237 Vem1 = Vai1;
238 vResp[10] = ULA10.Ativar(A[10], B[10], F, Vem1, out Vai1);
239 Vem1 = Vai1;
240 vResp[9] = ULA9.Ativar(A[9], B[9], F, Vem1, out Vai1);
241 Vem1 = Vai1;
242 vResp[8] = ULA8.Ativar(A[8], B[8], F, Vem1, out Vai1);
243 Vem1 = Vai1;
244 vResp[7] = ULA7.Ativar(A[7], B[7], F, Vem1, out Vai1);
245 Vem1 = Vai1;
246 vResp[6] = ULA6.Ativar(A[6], B[6], F, Vem1, out Vai1);
247 Vem1 = Vai1;
248 vResp[5] = ULA5.Ativar(A[5], B[5], F, Vem1, out Vai1);
249 Vem1 = Vai1;
250 vResp[4] = ULA4.Ativar(A[4], B[4], F, Vem1, out Vai1);
251 Vem1 = Vai1;
252 vResp[3] = ULA3.Ativar(A[3], B[3], F, Vem1, out Vai1);
253 Vem1 = Vai1;
254 vResp[2] = ULA2.Ativar(A[2], B[2], F, Vem1, out Vai1);
255 Vem1 = Vai1;
256 vResp[1] = ULA1.Ativar(A[1], B[1], F, Vem1, out Vai1);
```

2.12.6 Explicação dos Códigos da classe ULAs_Base

A classe "ULAs_Base" contém a versão básica de todas as ulas do projeto (1 bit, 8 bits, 16 bits e 32 bits), a classe ULA_1Bit_Base possui o decoder(em que tem a porta da soma, subtração, multiplicação e divisão) e um full adder, e a junção deles por meio das portas lógicas que compõem o circuito.

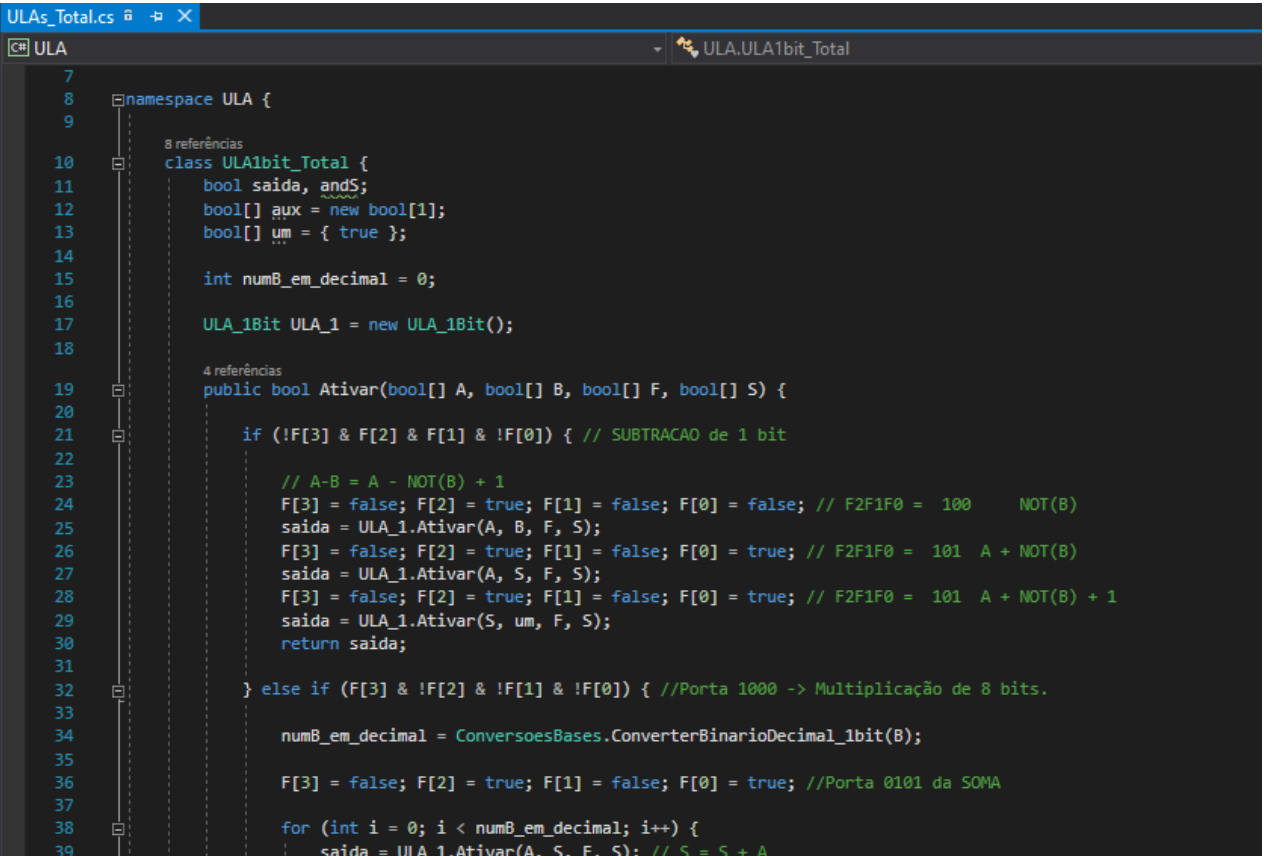
A classe "ULA_8bit" possui 8 instâncias da classe de ULA de 1 bit base, dessa forma é criado o método "Ativar" que recebe 2 vetores de boolean a serem calculados, um vetor booleano que define qual porta será utilizada para selecionar a operação a ser feita, e um vetor que irá conter a resposta da operação realizada entre esses 2 numeros, e por fim retorna o ultimo "Vai1" que posteriormente indicará o Overflow.

A classe "ULA_16bit" possui 16 instâncias da classe de ULA de 1 bit base, dessa forma é criado o método "Ativar" que recebe 2 vetores de boolean a serem calculados, um vetor booleano que define qual porta será utilizada para selecionar a operação a ser feita, e um vetor que irá conter a resposta da operação realizada

entre esses 2 numeros, e por fim retorna o ultimo "Vai1" que porteriormente indicará o Overflow.

A classe "ULA_32bit" possui 32 instâncias da classe de ULA de 1 bit base, dessa forma é criado o método "Ativar" que recebe 2 vetores de boolean a serem calculados, um vetor booleano que define qual porta será utilizada para selecionar a operação a ser feita, e um vetor que irá conter a resposta da operação realizada entre esses 2 numeros, e por fim retorna o ultimo "Vai1" que porteriormente indicará o Overflow.

2.12.7 Códigos da classe ULAs_Total



```
7
8 namespace ULA {
9
10     8 referências
11     class ULA1bit_Total {
12         bool saida, andS;
13         bool[] aux = new bool[1];
14         bool[] um = { true };
15
16         int numB_em_decimal = 0;
17
18         ULA_1Bit ULA_1 = new ULA_1Bit();
19
20         4 referências
21         public bool Ativar(bool[] A, bool[] B, bool[] F, bool[] S) {
22
23             if (!F[3] & F[2] & F[1] & !F[0]) { // SUBTRACAO de 1 bit
24
25                 // A-B = A - NOT(B) + 1
26                 F[3] = false; F[2] = true; F[1] = false; F[0] = false; // F2F1F0 = 100 NOT(B)
27                 saida = ULA_1.Ativar(A, B, F, S);
28                 F[3] = false; F[2] = true; F[1] = false; F[0] = true; // F2F1F0 = 101 A + NOT(B)
29                 saida = ULA_1.Ativar(A, S, F, S);
30                 F[3] = false; F[2] = true; F[1] = false; F[0] = true; // F2F1F0 = 101 A + NOT(B) + 1
31                 saida = ULA_1.Ativar(S, um, F, S);
32                 return saida;
33             } else if (F[3] & !F[2] & !F[1] & !F[0]) { //Porta 1000 -> Multiplicação de 8 bits.
34
35                 numB_em_decimal = ConversoesBases.ConverterBinarioDecimal_1bit(B);
36
37                 F[3] = false; F[2] = true; F[1] = false; F[0] = true; //Porta 0101 da SOMA
38
39                 for (int i = 0; i < numB_em_decimal; i++) {
40                     saida = ULA_1.Ativar(A, S, F, S); // S = S + A
41                 }
42             }
43         }
44     }
45 }
```

```

ULAs_Total.cs # -p X
ULA
36 F[3] = false; F[2] = true; F[1] = false; F[0] = true; //Porta 0101 da SOMA
37
38 for (int i = 0; i < numB_em_decimal; i++) {
39     saida = ULA_1.Ativar(A, S, F, S); // S = S + A
40 }
41
42 return saida;
43
44 } else if (F[3] & !F[2] & !F[1] & F[0]) { //Porta 1001 -> Divisão de 1 bit.
45     int numA_em_decimal = ConversoesBases.ConverterBinarioDecimal_1bit(A);
46     numB_em_decimal = ConversoesBases.ConverterBinarioDecimal_1bit(B);
47
48     int r = numA_em_decimal / numB_em_decimal;
49
50     ConversoesBases.ConverterDecimalBinario(S, r);
51     return saida;
52 } else {
53     saida = ULA_1.Ativar(A, B, F, S);
54     return saida;
55 }
56 }
57
58
59 8 referências
60 class ULA8bit_Total {
61     bool saida, andS;
62     bool[] aux = new bool[8];
63     bool[] um = { false, false, false, false, false, false, false, true };
64     bool[] zero = { false, false, false, false, false, false, false, false };
65
66     long numB_em_decimal = 0;
67
68     ULA_8Bit ULA_8 = new ULA_8Bit();

```

```

ULAs_Total.cs # -p X
ULA
66
67 ULA_8Bit ULA_8 = new ULA_8Bit();
68
69 4 referências
70 public bool Ativar(bool[] A, bool[] B, bool[] F, bool[] S) {
71
72     if (F[3] & F[2] & F[1] & F[0]) { // COMPARACAO de 8 bits
73         saida = ULA_8.Ativar(A, B, F, S);
74         andS = S[7] & S[6] & S[5] & S[4] & S[3] & S[2] & S[1] & S[0];
75         andS = PortasLogicas.AND(S[7], S[6], S[5], S[4], S[3], S[2], S[1], S[0]);
76         return andS;
77     } else if (!F[3] & F[2] & F[1] & !F[0]) { // SUBTRACAO de 8 bits
78
79         // A-B = A - NOT(B) + 1
80         F[3] = false; F[2] = true; F[1] = false; F[0] = false; // F2F1F0 = 100 NOT(B)
81         saida = ULA_8.Ativar(A, B, F, S);
82         F[3] = false; F[2] = true; F[1] = false; F[0] = true; // F2F1F0 = 101 A + NOT(B)
83         saida = ULA_8.Ativar(A, S, F, S);
84         F[3] = false; F[2] = true; F[1] = false; F[0] = true; // F2F1F0 = 101 A + NOT(B) + 1
85         saida = ULA_8.Ativar(S, um, F, S);
86         return saida;
87     } else if (F[3] & !F[2] & !F[1] & !F[0]) { //Porta 1000 -> Multiplicação de 8 bits.
88
89         numB_em_decimal = ConversoesBases.ConverterBinarioDecimal(B, 8);
90
91         F[3] = false; F[2] = true; F[1] = false; F[0] = true; //Porta 0101 da SOMA
92
93         for (int i = 0; i < numB_em_decimal; i++) {
94             saida = ULA_8.Ativar(A, S, F, S); // S = S + A
95         }
96
97         return saida;
98

```

```

98
99
100 } else if (F[3] & !F[2] & !F[1] & F[0]) { //Porta 1001 -> Divisão de 8 bits.
101     long numA_em_decimal = ConversoesBases.ConverterBinarioDecimal(A, 8);
102     numB_em_decimal = ConversoesBases.ConverterBinarioDecimal(B, 8);
103
104     long r = numA_em_decimal / numB_em_decimal;
105
106     ConversoesBases.ConverterDecimalBinario(S, r);
107     return saida;
108 } else {
109     saida = ULA_8.Ativar(A, B, F, S);
110     return saida;
111 }
112 }
113
114 8 referências
115 class ULA16bit_Total {
116     bool saida, andS;
117     bool[] aux = new bool[16];
118     bool[] um = { false, false, false, false, false, false, false, false, false, false, false, false, false, false, false, true };
119
120     long numB_em_decimal = 0;
121
122     ULA_16Bit ULA_16 = new ULA_16Bit();
123
124     4 referências
125     public bool Ativar(bool[] A, bool[] B, bool[] F, bool[] S) {
126
127         if (F[3] & F[2] & F[1] & F[0]) { // COMPARACAO de 16 bits
128             saida = ULA_16.Ativar(A, B, F, S);
129             andS = S[15] & S[14] & S[13] & S[12] & S[11] & S[10] & S[9] & S[8] & S[7] & S[6] & S[5] & S[4] & S[3] & S[2] & S[1] & S[0];
130             andS = PortasLogicas.AND(S[15], S[14], S[13], S[12], S[11], S[10], S[9], S[8], S[7], S[6], S[5], S[4], S[3], S[2], S[1], S[0]);
131             return andS;
132         } else if (!F[3] & F[2] & F[1] & !F[0]) { // SUBTRACAO de 8 bits

```

```

129         return andS;
130     } else if (!F[3] & F[2] & F[1] & !F[0]) { // SUBTRACAO de 8 bits
131
132         // A-B = A - NOT(B) + 1
133         F[3] = false; F[2] = true; F[1] = false; F[0] = false; // F2F1F0 = 100 NOT(B)
134         saida = ULA_16.Ativar(A, B, F, S);
135         F[3] = false; F[2] = true; F[1] = false; F[0] = true; // F2F1F0 = 101 A + NOT(B)
136         saida = ULA_16.Ativar(A, S, F, S);
137         F[3] = false; F[2] = true; F[1] = false; F[0] = true; // F2F1F0 = 101 A + NOT(B) + 1
138         saida = ULA_16.Ativar(S, um, F, S);
139         return saida;
140
141     } else if (F[3] & !F[2] & !F[1] & !F[0]) { //Porta 1000 -> Multiplicação de 8 bits.
142
143         numB_em_decimal = ConversoesBases.ConverterBinarioDecimal(B, 16);
144
145         F[3] = false; F[2] = true; F[1] = false; F[0] = true; //Porta 0101 da SOMA
146
147         for (int i = 0; i < numB_em_decimal; i++) {
148             saida = ULA_16.Ativar(A, S, F, S); // S = S + A
149         }
150
151         return saida;
152
153     } else if (F[3] & !F[2] & !F[1] & F[0]) { //Porta 1001 -> Divisão de 16 bits.
154         long numA_em_decimal = ConversoesBases.ConverterBinarioDecimal(A, 16);
155         numB_em_decimal = ConversoesBases.ConverterBinarioDecimal(B, 16);
156
157         long r = numA_em_decimal / numB_em_decimal;
158
159         ConversoesBases.ConverterDecimalBinario(S, r);
160         return saida;
161     } else {
162         saida = ULA_16.Ativar(A, B, F, S);
163         return saida;

```

```

162         saida = ULA_16.Ativar(A, B, F, S);
163         return saida;
164     }
165 }
166
167
168 8 referências
169 class ULA32bit_Total {
170     bool saida, andS;
171     bool[] aux = new bool[32];
172     bool[] um = { false, false, false, false, false, false, false, false, false, false, false, false, false, false, false, false,
173                 false, false, false, false, false, false, false, false, false, false, false, false, true };
174
175     long numB_em_decimal = 0;
176
177     ULA_32Bit ULA_32 = new ULA_32Bit();
178
179 4 referências
180 public bool Ativar(bool[] A, bool[] B, bool[] F, bool[] S) {
181
182     if (F[3] & F[2] & F[1] & F[0]) { // COMPARACAO de 16 bits
183         saida = ULA_32.Ativar(A, B, F, S);
184         andS = S[31] & S[30] & S[29] & S[28] & S[27] & S[26] & S[25] & S[24] & S[23] & S[22] & S[21] & S[20] & S[19] & S[18] & S[17]
185             & S[16] & S[15] & S[14] & S[13] & S[12] & S[11] & S[10] & S[9] & S[8] & S[7] & S[6] & S[5] & S[4] & S[3] & S[2] &
186             S[1] & S[0];
187         andS = PortasLogicas.AND(S[31], S[30], S[29], S[28], S[27], S[26], S[25], S[24], S[23], S[22], S[21], S[20], S[19], S[18],
188             S[17], S[16], S[15], S[14], S[13], S[12], S[11], S[10], S[9], S[8], S[7], S[6], S[5], S[4], S[3], S[2], S[1], S[0]);
189         return andS;
190     } else if (!F[3] & F[2] & F[1] & !F[0]) { // SUBTRACAO de 8 bits
191
192         // A-B = A - NOT(B) + 1
193         F[3] = false; F[2] = true; F[1] = false; F[0] = false; // F2F1F0 = 100 NOT(B)
194         saida = ULA_32.Ativar(A, B, F, S);
195         F[3] = false; F[2] = true; F[1] = false; F[0] = true; // F2F1F0 = 101 A + NOT(B)
196         saida = ULA_32.Ativar(A, S, F, S);
197     }
198 }

```

```

193     F[3] = false; F[2] = true; F[1] = false; F[0] = true; // F2F1F0 = 101 A + NOT(B)
194     saida = ULA_32.Ativar(A, S, F, S);
195     F[3] = false; F[2] = true; F[1] = false; F[0] = true; // F2F1F0 = 101 A + NOT(B) + 1
196     saida = ULA_32.Ativar(S, um, F, S);
197     return saida;
198 }
199
200 } else if (F[3] & !F[2] & !F[1] & !F[0]) { //Porta 1000 -> Multiplicação de 8 bits.
201
202     numB_em_decimal = ConversoesBases.ConverterBinarioDecimal(B, 32);
203
204     F[3] = false; F[2] = true; F[1] = false; F[0] = true; //Porta 0101 da SOMA
205
206     for (int i = 0; i < numB_em_decimal; i++) {
207         saida = ULA_32.Ativar(A, S, F, S); // S = S + A
208     }
209
210     return saida;
211 }
212
213 } else if (F[3] & !F[2] & !F[1] & F[0]) { //Porta 1001 -> Divisão de 32 bits.
214     long numA_em_decimal = ConversoesBases.ConverterBinarioDecimal(A, 32);
215     numB_em_decimal = ConversoesBases.ConverterBinarioDecimal(B, 32);
216
217     long r = numA_em_decimal / numB_em_decimal;
218
219     ConversoesBases.ConverterDecimalBinario(S, r);
220     return saida;
221 }
222
223 } else {
224     saida = ULA_32.Ativar(A, B, F, S);
225     return saida;
226 }
227 }
228 }

```


2.12.7 Explicação dos Códigos da classe ULAs_Total

A classe "ULA1bit_Total" possui o código final da ULA, em que é realizado a soma, subtração (por meio da expressão $A + \text{NOT}(B) + 1$), a multiplicação (por meio de N somas), e a divisão.

O mesmo se aplica para as outras classes "ULA8bit_Total", "ULA16bit_Total" e "ULA32bit_Total", diferenciando apenas a quantidade de bits, cada ULA TOTAL possui uma instância de ULA diferente, por exemplo a de 8 bit total tem uma instancia da ula de 8 bits base e assim por diante.

3. CONCLUSÃO

Considerando as medidas e análises realizadas neste trabalho, percebemos que para a criação dos programas, foram utilizados recursos práticos e teóricos desenvolvidos em salas de aula de arquitetura.

Neste trabalho, implementamos a ULA (unidade lógica e aritmética) com o seu principal objetivo de fazer as operações aritméticas de somar, subtrair, multiplicar e dividir de ulas de 1, 8, 16 e 32 bits. Foi implementado um sistema orientado a objeto com o objetivo de visualizar e entender melhor os programas.

Para a criação a ULA, utilizamos diversos sites e explicações do professor orientador para ajudar no entendimento da ULA, de como funciona passo a passo e como a ULA é utilizada. Com base no que foi nos apresentados, foi desenvolvido um programa em C#.

Conclui-se que nós aperfeiçoamos nossos conhecimentos em arquitetura de computadores, a criação de programas orientados a objeto e percebemos que este modelo de análise e programação é um método mais eficiente e mais empregado para o desenvolvimento de softwares.

REFERÊNCIAS

CANAL TI. **Como funciona um processador?** (CPU, UC, ULA, Registradores). Disponível em: <<https://www.canaltt.com.br/sistemas-operacionais/como-funciona-um-processador-cpu-uc-ula-registradores/>> Acesso em: 21 jun. 2019.

DEVMEDIA. **Programação Orientada a Objetos**. Disponível em: <<https://www.devmedia.com.br/os-4-pilares-da-programacao-orientada-a-objetos/9264>> Acesso em: 20 jun. 2019.

DICAS DE PROGRAMAÇÃO. **As 10 conversões numéricas mais utilizadas na computação**. Disponível em: <<https://dicasdeprogramacao.com.br/as-10-conversoes-numericas-mais-utilizadas-na-computacao/>> Acesso em: 21 jun. 2019.

UNIDADE LÓGICA E ARITMÉTICA. In: **Wikipédia: a enciclopédia livre**. Disponível em: <https://pt.wikipedia.org/wiki/Unidade_l%C3%B3gica_e_aritm%C3%A9tica> Acesso em: 20 nov. 2019.

WIKILIVROS. **Introdução à arquitetura de computadores/ as operações lógicas aritméticas**. Disponível em: <https://pt.wikibooks.org/wiki/Introdu%C3%A7%C3%A3o_%C3%A0_Arquitetura_de_Computadores/As_Opera%C3%A7%C3%B5es_da_Unidade_L%C3%B3gica_Aritm%C3%A9tica> Acesso em: 20 nov. 2019.